Under the Hood: The Code That Runs the Experiment

A Line-by-Line Walkthrough of the Python Behind Phases 1 and 2

How raw code becomes a money question, an answer, and a fitted number

J. Blakeslee, RAND School of Public Policy

AI Sprint Practicum · July 2026

Who this is for: a reader who can follow a little Python (a loop, a function, a dictionary) but who has never seen this project. Every code block below is the real code from the repository, copied unchanged. I explain each block in plain words right underneath, and I box every piece of coding or statistics jargon the first time it appears. If you only remember one thing: the question the model reads is assembled by ordinary Python string formatting, and I can point at the exact line where the wording lives.

1. Orientation: One Pipeline, Three Stages

The whole project is a pipeline that turns a research question into a number. It runs in three stages, and each stage lives in its own set of files. Keep this map in your head; everything below is a zoom-in on one of these three boxes.

Stage 1, design the questions. I write code that manufactures hundreds of small money choices and turns each one into the exact block of text the model will read. This lives in `src/design.py` (Phase 1) and `src/design_phase2.py` (Phase 2, which adds the gain/loss wording). No AI is involved yet; this stage is pure bookkeeping that produces a list of questions.

Stage 2, ask the model and record answers. I write code that takes that list of questions, sends each one to the AI model, forces a clean "A" or "B" back, and writes one row of a spreadsheet per answer. This lives in `src/harness.py`. The output is a CSV file: one row per choice, fully labeled.

Stage 3, fit the math to the answers. I write code that reads the CSV and searches for the handful of numbers (risk aversion, probability weighting, loss aversion) that best explain the pattern of choices. This lives in `analysis/estimate_crra.py` and its richer cousin `analysis/estimate_cpt.py`. The output is a small table of estimates.

Design (`design.py`, `design_phase2.py`) makes the questions. The harness (`harness.py`) asks the model and saves the answers. The estimators (`estimate_crra.py`, `estimate_cpt.py`) read the answers and fit the economics. Three stages, three folders, one direction of flow: questions → answers → numbers.

2. How I Run It

Before dissecting the code, here is how it is actually launched from a terminal. These commands come straight from the project `README.md`.

```
# validate the whole pipeline against a simulated agent (no API key needed)
python src/harness.py --dry-run --reps 3
python analysis/estimate_crra.py data/phase1_dryrun.csv
# -> should recover r ~= 0.5, mu ~= 0.15

# live pilot against Claude (needs ANTHROPIC_API_KEY exported)
python src/harness.py --pilot 40
python analysis/estimate_crra.py data/phase1_claude-haiku-4-5-20251001.csv

# full Phase 1 run (~1,200 calls at default settings)
python src/harness.py --reps 3
```

Read the block top to bottom. The first pair of commands is a rehearsal that costs nothing. The second pair is a small paid test, forty questions, just to confirm the live wiring works. The last command is the real run, roughly 1,200 questions. Notice that every "run" command produces a CSV, and every "estimate" command reads a CSV. That is the pipeline of Section 1, expressed as four shell commands.

DEFINITION • Dry run

A **dry run** is a full rehearsal of the pipeline that never touches the paid AI. Instead of the real model, the code substitutes a fake decider whose answers I already understand (see `SimulatedCRRAAgent` in Section 5). Because I know exactly how the fake decider thinks, I can run the entire loop, design the questions, collect answers, fit the model, and then check that the estimator recovers the fake decider's known settings. If it does, the plumbing is sound and I can safely spend money on the real model. A dry run needs no **API key** and costs nothing.

Why rehearse against a fake before paying for the real thing? Two reasons. First, bugs are cheaper to find here: if the estimator cannot even recover an answer I built by hand, the problem is my code, not the AI. Second, it is a correctness proof for the estimator. The comment `# -> should recover r ~= 0.5, mu ~= 0.15` is a promise: I built the fake decider with risk aversion $r = 0.5$ and noise $\mu = 0.15$, so a correct estimator must read those same numbers back out of the simulated choices. This is the software equivalent of calibrating a scale with a known weight before weighing something unknown.

2.1 What the resulting CSV looks like

Each run appends rows to a CSV (comma-separated values) file, one row per choice. A row carries every design fact about the question plus the model's answer and some run metadata. Schematically, the key columns are:

```
trial_id, gamble_id, sure, hi, p, ev_ratio, template_id, safe_first, rep,
response_mode, model, temperature, seed, letter, chose_gamble, raw,
```

```
discarded, latency_s, ts_utc
```

The load-bearing columns for the analysis are `sure` (the guaranteed dollar amount), `hi` and `p` (the gamble's prize and its probability), `safe_first` (which option was listed first, for the position check), `template_id` (which of the five wordings was used), and above all `chose_gamble`, a 1 if the model took the gamble and a 0 if it took the sure thing. Stage 3 reads `sure`, `hi`, `p`, and `chose_gamble` and asks: what risk attitude would make this pattern of 1s and 0s least surprising?

3. Stage 1: Designing the Questions (`design.py`)

This file never talks to an AI. Its only job is to manufacture questions. I walk through it in the order the data flows: first the object that holds one gamble, then the function that mass-produces gambles, then the templates that hold the wording, and finally the function that turns a gamble plus a template into the exact text the model sees.

3.1 The `Gamble`: one object that holds one choice

```python
@dataclass(frozen=True)
class Gamble:
    """One binary choice: a sure amount vs. a two-outcome gamble (hi with prob p, else 0)."""
    gamble_id: int
    sure: int       # sure payoff, dollars
    hi: int         # gamble's high payoff, dollars
    p: float        # probability of the high payoff

    @property
    def ev_gamble(self) -> float:
        return self.p * self.hi

    @property
    def ev_ratio(self) -> float:
        """EV(gamble) / sure. >1 means the gamble is actuarially favorable."""
        return self.ev_gamble / self.sure
```

A `Gamble` bundles four facts: an id number, the sure amount, the gamble's high payoff, and the probability of that payoff. The two `@property` methods are small computed conveniences. `ev_gamble` is the expected value of the gamble, $p \times \text{hi}$: the average dollars it pays over many plays. `ev_ratio` divides that by the sure amount, giving a single number that says whether the gamble is a good deal on paper. Above 1 means the gamble's average payout beats the sure thing.

DEFINITION • Dataclass

A **dataclass** is a lightweight Python container for a fixed set of named fields. Writing `@dataclass` above a class tells Python to generate the boilerplate for me: a constructor, a readable printout, and equality checks, all for free. The line `sure: int` declares a field named `sure` that holds a whole number. `frozen=True` makes each `Gamble` immutable: once created it cannot be changed, which prevents accidental edits from corrupting a question halfway

through a run. Think of a dataclass as a labeled box with a fixed number of slots.

3.2 Manufacturing gambles, and why the numbers are odd

Now the factory. Two functions do the work: a helper that draws a non-round number, and the generator that builds a whole grid of gambles.

```
def _non_round(rng: random.Random, lo: int, hi: int) -> int:
    """Draw an integer that is not a multiple of 5 (contamination safety:
    avoids the round numbers canonical instruments use)."""
    while True:
        x = rng.randint(lo, hi)
        if x % 5 != 0:
            return x


def generate_gambles(n: int = 40, seed: int = 42) -> list[Gamble]:
    """Procedurally generate a grid of gambles spanning EV ratios ~0.75-1.7.

    The spread of EV ratios is what identifies risk aversion: a risk-neutral
    agent switches to the gamble as soon as ev_ratio > 1; a risk-averse agent
    demands a premium. Probabilities are jittered to two decimals so no row
    matches a textbook menu.
    """
    rng = random.Random(seed)
    gambles = []
    for i in range(n):
        sure = _non_round(rng, 23, 187)
        # jittered, non-canonical probability in [0.19, 0.83]
        p = round(rng.uniform(0.19, 0.83), 2)
        if p in (0.25, 0.50, 0.75):  # avoid the textbook values exactly
            p += 0.01
        # target EV ratio spread; log-uniform so both sides of 1.0 are covered
        ratio = 0.75 * (1.7 / 0.75) ** rng.random()
        hi = max(int(round(sure * ratio / p)), sure + 1)
        if hi % 5 == 0:
            hi += 1
        gambles.append(Gamble(gamble_id=i, sure=sure, hi=hi, p=p))
    return gambles
```

This is **procedural generation**: instead of typing out forty questions by hand, I write a recipe and let the computer roll them. The loop runs `n` times (forty by default). Each pass draws a sure amount between $23 and $187, draws a probability between 0.19 and 0.83, picks a target `ev_ratio` somewhere in the range 0.75 to 1.7, and then back-solves for the prize `hi` that hits that ratio. The line `ratio = 0.75 * (1.7 / 0.75) ** rng.random()` spreads the ratios so that some gambles are bad deals and some are good deals, which is exactly the spread needed to see where the model draws its line.

The odd-number trick is the interesting part, and it is a safety measure. Textbook risk experiments use round, memorable numbers: a 50 percent chance of $100. If I reused those, the model might recite a memorized textbook answer instead of actually deciding. So `_non_round` refuses any multi-

ple of 5, the code bumps a probability off the canonical 0.25/0.50/0.75 values, and `hi` is nudged off any multiple of 5. The result is deliberately awkward amounts like $58 and $116 that no textbook would print, which forces a genuine computation rather than a recital.

DEFINITION • Seed

A **seed** is a starting number for the random-number generator. Computers do not produce true randomness; they produce a fixed, reproducible sequence that only looks random, and the seed picks which sequence. `random.Random(seed)` creates a generator pinned to that sequence, so `generate_gambles(40, seed=42)` draws the exact same forty gambles every single time anyone runs it, on any machine. That is what makes the experiment reproducible: a colleague who runs my code with seed 42 sees byte-for-byte the same questions I did.

`generate_gambles` rolls forty money choices from a recipe rather than typing them by hand. It spreads them across good and bad deals so the model's switching point is visible, and it forces every dollar amount and probability to be awkward and non-round so the model cannot parrot a memorized textbook answer. The `seed` makes the whole roll reproducible.

3.3 Where the wording lives: the `TEMPLATES` dictionary

Here is the single most important block in the whole design stage. This is where a gamble stops being numbers and becomes English. Everything the model reads is built from these five strings.

```
TEMPLATES: dict[int, str] = {
    0: ("You must pick one of two payment options.\n"
        "Option A: {first}\n"
        "Option B: {second}\n"
        "Pick the option you prefer."),
    1: ("Two offers are on the table and you can accept exactly one.\n"
        "Offer A: {first}\n"
        "Offer B: {second}\n"
        "Which offer do you accept?"),
    2: ("Consider the following pair of alternatives and select one.\n"
        "Alternative A: {first}\n"
        "Alternative B: {second}\n"
        "Select the alternative you would rather have."),
    3: ("You are entitled to one of two payouts.\n"
        "Payout A: {first}\n"
        "Payout B: {second}\n"
        "Choose the payout you want."),
    4: ("A counterparty gives you a one-time decision between two deals.\n"
        "Deal A: {first}\n"
        "Deal B: {second}\n"
        "Decide which deal you take."),
}
```

DEFINITION • Dictionary

A Python **dictionary** maps keys to values, like a lookup table. `TEMPLATES` maps an integer key (0 through 4) to a string value (a wording). Later, `TEMPLATES[self.template_id]` looks up one wording by its number. The declaration `dict[int, str]` just says "keys are integers, values are strings."

`TEMPLATES` holds five semantically equivalent phrasings of the same underlying question. Read them: "payment options," "offers," "alternatives," "payouts," "deals." The economics is identical in all five; only the register differs. This is a deliberate experimental control. If a result showed up under only one phrasing, it would be a quirk of that phrasing, not a real preference, so I log which template produced each choice and later let the analysis check that the result holds across all five.

The two pieces in curly braces, `{first}` and `{second}`, are **placeholders**. They are empty slots that Python will fill in later with the actual descriptions of the two options. The template itself is agnostic about which option is the safe one and which is the gamble; it just knows there is a first option and a second option. Deciding what goes in each slot is the job of the next two functions.

3.4 Describing the two options

```
106 def describe_safe(g: Gamble) -> str:
107     return f"receive ${g.sure} with certainty."
108
109
110 def describe_risky(g: Gamble) -> str:
111     pct = round(g.p * 100)
112     return (f"receive ${g.hi} with {pct}% probability, "
113             f"and $0 with {100 - pct}% probability.")
```

These two small functions turn the numeric fields of a `Gamble` into English phrases. Given a gamble with `sure=58`, `describe_safe` returns the string `receive $58 with certainty.` Given `hi=116, p=0.66`, `describe_risky` converts the probability to a percentage and returns:

```
receive $116 with 66% probability, and $0 with 34% probability.
```

The `f"..."` syntax is an **f-string**: Python substitutes the value of each braced expression into the text. Note the literal `$` and `%` characters here; they are just ordinary characters inside these strings, which matters when the phrases get slotted into a template.

3.5 `Trial.prompt`: where a gamble becomes the exact question

A `Trial` is one fully-specified question: a particular gamble, in a particular wording, in a particular order, on a particular repetition. Its `prompt` method is the hinge of the entire design stage. This is the function that produces the literal text the model reads.

```
120 @dataclass
121 class Trial:
122     trial_id: int
123     gamble_id: int
```

```
    sure: int
    hi: int
    p: float
    ev_ratio: float
    template_id: int
    safe_first: bool     # counterbalanced: True -> A is the sure amount
    rep: int
    response_mode: str   # 'tool' (forced structured output) or 'text' (validation arm)

    def prompt(self) -> str:
        g = Gamble(self.gamble_id, self.sure, self.hi, self.p)
        safe, risky = describe_safe(g), describe_risky(g)
        first, second = (safe, risky) if self.safe_first else (risky, safe)
        return TEMPLATES[self.template_id].format(first=first, second=second)

    @property
    def gamble_letter(self) -> str:
        """Which letter maps to the risky option in this trial."""
        return "B" if self.safe_first else "A"
```

Walk through `prompt` one line at a time, because this is where the question is born.

1. `g = Gamble(...)` rebuilds the gamble object from the trial's stored fields.
2. `safe, risky = describe_safe(g), describe_risky(g)` produces the two English phrases: one for the sure thing, one for the gamble.
3. `first, second = (safe, risky) if self.safe_first else (risky, safe)` is the counterbalancing switch. If `safe_first` is true, the sure phrase goes into the first slot and the gamble into the second; if it is false, they swap. This is how the same gamble gets presented in both orders.
4. `return TEMPLATES[self.template_id].format(first=first, second=second)` looks up the chosen wording and calls `.format(...)`, which drops the two phrases into the `{first}` and `{second}` slots. The returned string is the exact block of text the model will see.

The `gamble_letter` property records the answer key. If the sure option was listed first, then the gamble is option "B"; otherwise it is "A." The harness later compares the model's letter to this to decide whether the model chose the gamble. That is how a raw "A" or "B" becomes the clean `chose_gamble` column.

`Trial.prompt` is the exact moment a gamble turns into a question. It builds the two option descriptions, decides which one is listed first (counterbalancing), and slots them into one of the five wordings. Whatever this function returns is character-for-character what the model reads. If you want to know what the model actually saw, you read this function.

3.6 `build_trials`: every gamble, every wording, both orders

Finally, the function that assembles the full question list.

```python
def build_trials(gambles: list[Gamble], reps: int = 3, seed: int = 42,
                 text_arm_share: float = 0.10) -> list[Trial]:
    """Full factorial: gambles x templates x both orders x reps.

    ~10% of trials use plain-text elicitation instead of forced tool output,
    so the two response modes can be compared on the same instruments
    (design commitment: validate that output forcing does not change choices).
    """
    rng = random.Random(seed + 1)
    trials = []
    tid = 0
    for g in gambles:
        for template_id in TEMPLATES:
            for safe_first in (True, False):
                for rep in range(reps):
                    mode = "text" if rng.random() < text_arm_share else "tool"
                    trials.append(Trial(
                        trial_id=tid, gamble_id=g.gamble_id,
                        sure=g.sure, hi=g.hi, p=g.p,
                        ev_ratio=round(g.ev_ratio, 4),
                        template_id=template_id, safe_first=safe_first,
                        rep=rep, response_mode=mode,
                    ))
                    tid += 1
    rng.shuffle(trials)  # randomize run order so drift/rate-limits don't confound conditions
    return trials
```

The four nested `for` loops are a **full factorial**: every combination of every factor. For each of the forty gambles, for each of the five templates, for each of the two orders (safe first or risky first), for each of the three repetitions, the code builds one `Trial`. That is $40 \times 5 \times 2 \times 3 = 1{,}200$ questions, which is where the "~1,200 calls" figure comes from.

Two details deserve a note. This line,

```python
mode = "text" if rng.random() < text_arm_share else "tool"
```

sends about 10 percent of questions down a plain-text path instead of the forced-tool path (Section 5 explains both). This is a validation arm: it lets me check that forcing the model to answer through a tool does not itself change the answers. And the final `rng.shuffle(trials)` scrambles the run order. Without it, all the "template 0" questions would run first, then all the "template 1" questions, and if the model or the network drifted over the run, that drift would masquerade as a template effect. Shuffling breaks any link between when a question runs and which condition it belongs to.

`build_trials` produces the master list of about 1,200 questions by taking every gamble, in every wording, in both orders, repeated three times. It marks about a tenth of them for a plain-text check, then shuffles the whole list so run order cannot be mistaken for a real effect.

4. Stage 1b: The Phase 2 Wording (`design_phase2.py`)

Phase 2 reuses the entire Phase 1 machinery and adds one thing: framing. The same underlying gamble is now dressed in different words to test whether wording alone moves the model's choice. All the gamble generation, the counterbalancing, and the five paraphrase templates carry over unchanged; the new code is one function that produces the framed option descriptions.

4.1 `framed_options`: one gamble, four sets of words

```
def framed_options(frame: str, s: int, h: int, p: float) -> tuple[str, str, str]:
    """Return (endowment_line, safe_description, target_description) for a frame.

    The 'target' is the non-safe slot: the gamble in neutral/gain/loss frames, or the
    larger certain amount in the dominant control.
    """
    if frame == "neutral":
        endow = ""
        safe = f"receive ${s} for certain"
        target = (f"receive ${h} with {_pct(p)}% probability, "
                  f"and $0 with {100 - _pct(p)}% probability")
    elif frame == "gain":
        endow = "You are starting from $0. "
        safe = f"a certain gain of ${s}"
        target = (f"a {_pct(p)}% chance to gain ${h} "
                  f"and a {100 - _pct(p)}% chance to gain nothing")
    elif frame == "mixed":
        # anchor at the sure amount: the gamble straddles the reference (a gain of
        # $(h-s) above it or a loss of $s below it), which is what identifies loss
        # aversion. Terminal wealth still matches the gain frame exactly.
        endow = f"You have been given ${s} up front. "
        safe = f"keep your ${s} with no change"
        target = (f"a {_pct(p)}% chance to rise to ${h} (a gain of ${h - s}) "
                  f"and a {100 - _pct(p)}% chance to drop to $0 (a loss of ${s})")
    elif frame == "dominant":  # positive control: h is a strictly larger certain amount
        endow = ""
        safe = f"receive ${s} for certain"
        target = f"receive ${h} for certain"
    else:
        raise ValueError(f"unknown frame: {frame}")
    return endow, safe, target
```

This function is the heart of Phase 2. It takes a frame name and the gamble's numbers, and returns three strings: an optional opening line (`endow`), a description of the safe option, and a description of the "target" (the gamble, or in the control, the larger sure amount). Walk the four frames:

- **neutral** reproduces the Phase 1 wording exactly: a plain "receive $s for certain" versus "receive $h with some percent probability." No opening line, no gain-or-loss language. This is the baseline for comparison.
- **gain** states an explicit starting point of $0 and frames both options as gains: "a certain gain of $s" versus "a chance to gain $h." Everything is worded as money coming in.
- **mixed** is the crucial frame, and it is the only one that says the word "loss." It hands the

reader $s up front, so the safe option becomes "keep your $s with no change," and the gamble is worded as "a chance to rise to $h (a gain of $(h−s)) and a chance to drop to $0 (a loss of $s)." The final dollar amounts are identical to the gain frame; only the framing changed. This is the wording that produces the framing result.

- **dominant** is a positive control. It offers "$s for certain" versus a strictly larger "$h for certain." Any decider that reads the numbers at all should pick the bigger amount every time, so this frame proves the model can read magnitudes when nothing distracts it.

The key idea to hold onto: the same underlying gamble, the same terminal wealth, becomes visibly different text depending on the frame. The gain and mixed frames leave the model with identical money, yet the mixed frame introduces the single word "loss." If the model's choices differ between them, the words alone moved the decision.

4.2 `Phase2Trial.prompt`: same slotting, framing-aware

```
111     def prompt(self) -> str:
112         endow, safe, target = framed_options(self.frame, self.sure, self.hi, self.p)
113         first, second = (safe, target) if self.safe_first else (target, safe)
114         return PHASE2_TEMPLATES[self.template_id].format(endow=endow, first=first, second=
    ↪ second)
```

This mirrors `Trial.prompt` from Phase 1, with two changes. It calls `framed_options` to get the framed descriptions, and the Phase 2 templates carry an extra `{endow}` slot for the opening line ("You have been given $s up front."). The counterbalancing swap and the final `.format(...)` are the same idea as before: decide which description is first, then drop everything into the chosen wording.

4.3 Seeing one prompt per frame

The file ends with a small self-test that prints one assembled prompt for each frame, so I can eyeball the actual text.

```
163 if __name__ == "__main__":  # quick visual check of one trial per frame
164     g = generate_gambles(3, seed=42)[0]
165     for frame in ("neutral", "gain", "mixed", "dominant"):
166         anchor = g.sure if frame == "mixed" else 0
167         h = g.sure + 30 if frame == "dominant" else g.hi
168         t = Phase2Trial(0, 0, frame, anchor, g.sure, h, g.p if frame != "dominant" else 1.0,
169                         1.0, 0, True, 0, "tool")
170         print(f"\n===== {frame} =====\n{t.prompt()}")
```

The guard `if __name__ == "__main__":` means "run this block only when the file is executed directly, not when it is imported." Running `python src/design_phase2.py` prints four blocks of text, one per frame, each showing exactly what the model would read. It is a cheap sanity check that the wording came out as intended before spending anything on live calls.

Phase 2 keeps all of Phase 1 and adds `framed_options`, which re-describes one gamble under four frames. The neutral frame matches Phase 1; the gain frame words everything as gains; the mixed frame is the only one that says "loss" and is what drives the framing result; the dominant frame is a read-the-numbers control. `Phase2Trial.prompt` slots these into the same five paraphrases.

5. Stage 2: Asking the Model (`harness.py`)

Now the questions exist as a list. This file sends each one to the model and records the answer. I cover four pieces: how the API key is read, how the model is forced to answer cleanly, the fake decider used for dry runs, and the main loop that writes the CSV.

5.1 Reading the API key without hard-coding it

```
37 def load_dotenv() -> None:
38     """Read KEY=VALUE lines from the project-root .env (gitignored) into the
39     environment, so the API key never lives in code or shell profiles."""
40     env_file = PROJECT_ROOT / ".env"
41     if not env_file.exists():
42         return
43     for line in env_file.read_text().splitlines():
44         line = line.strip().removeprefix("export ").strip()
45         if line and not line.startswith("#") and "=" in line:
46             k, v = line.split("=", 1)
47             os.environ.setdefault(k.strip(), v.strip().strip('"').strip("'"))
```

DEFINITION • API and API key

An **API** (Application Programming Interface) is a structured way for one program to send requests to another. Here my code sends a money question to Anthropic's servers and gets back the model's answer. An **API key** is a secret password that proves I am allowed to make those requests and tells the provider whose account to bill. Because it is a secret, it must never be written into the code or committed to version control; anyone who reads the key can spend money as me.

`load_dotenv` reads a file named `.env` that sits in the project root and is deliberately excluded from version control ("gitignored"). Each line looks like `ANTHROPIC_API_KEY=sk-...`. The function walks the lines, skips blanks and comments, splits each on the first `=`, and loads the result into the program's environment. The key thus lives in one local, private file, never in the code. `os.environ.setdefault` means "only set this if it is not already set," so a key already present in the shell wins and is not overwritten.

5.2 Forcing a clean answer: the choice tool

Left to its own devices, a chat model might answer a money question with a paragraph of reasoning. I do not want a paragraph; I want a single letter I can tally. The solution is tool use.

```
CHOICE_TOOL = {
    "name": "submit_choice",
    "description": "Submit your choice between the two options.",
    "input_schema": {
        "type": "object",
        "properties": {"choice": {"type": "string", "enum": ["A", "B"]}},
        "required": ["choice"],
    },
}
```

DEFINITION • Tool use

Tool use lets a language model return a structured, machine-readable action instead of free text. I define a "tool" named `submit_choice` whose only input is a field called `choice` that must be either "A" or "B" (that is what `"enum": ["A", "B"]` enforces). When I require the model to answer through this tool, its reply is guaranteed to be a clean A or B, with no essay to parse and no ambiguity about what it meant. It converts an open-ended chat into a crisp multiple-choice button press.

5.3 `ClaudeAgent.choose`: one question, one fresh conversation

```
class ClaudeAgent:
    """Live elicitation against the Anthropic API. Fresh context per call
    (independence); forced tool output on 'tool' trials, first-token parse
    on 'text' trials (the validation arm)."""

    def __init__(self, model: str, temperature: float):
        import anthropic
        self.client = anthropic.Anthropic()
        self.model = model
        self.temperature = temperature

    def choose(self, trial: Trial) -> tuple[str | None, str]:
        prompt = trial.prompt()
        if trial.response_mode == "tool":
            resp = self.client.messages.create(
                model=self.model, max_tokens=64, temperature=self.temperature,
                tools=[CHOICE_TOOL],
                tool_choice={"type": "tool", "name": "submit_choice"},
                messages=[{"role": "user", "content": prompt}],
            )
            for block in resp.content:
                if block.type == "tool_use":
                    return block.input.get("choice"), str(block.input)
            return None, str(resp.content)
        # plain-text validation arm
        resp = self.client.messages.create(
```

```
            model=self.model, max_tokens=5, temperature=self.temperature,
            messages=[{"role": "user", "content":
                       prompt + "\nRespond with only the letter A or B."}],
        )
        text = resp.content[0].text.strip() if resp.content else ""
        letter = text[:1].upper()
        return (letter if letter in ("A", "B") else None), text
```

The `choose` method is the live counterpart to a single question. It starts by calling `trial.prompt()`, the exact function from Section 3.6, to build the question text. Then it branches on the response mode.

On a "tool" trial it calls the API with the choice tool attached and this argument,

```
tool_choice={"type": "tool", "name": "submit_choice"}
```

which forces the model to answer through the tool. It then scans the reply for the tool-use block and returns the chosen letter. On a "text" trial (the 10 percent validation arm) it instead appends "Respond with only the letter A or B" and reads the first character of the plain reply. Either way, `choose` returns a tuple: the letter (or `None` if the model somehow failed to give a clean one) and a raw copy of what came back, for logging.

The single most important design choice here is invisible unless you look for it. Each call passes `messages=[{"role": "user", "content": prompt}]`, a message list containing only this one question. There is no history. Every question is a brand-new conversation with a model that remembers nothing of the previous questions. This independence is what lets me treat the 1,200 answers as 1,200 clean data points rather than one long, self-influencing dialogue.

`ClaudeAgent.choose` builds one question with `trial.prompt`, sends it to the model, and forces a clean A or B back, either through the choice tool or, for a tenth of trials, by asking for a bare letter. Crucially, each question is its own fresh conversation, so answers stay independent.

5.4 The fake decider for dry runs

```
class SimulatedCRRAAgent:
    """Synthetic subject for --dry-run: a CRRA expected-utility maximizer with
    logit choice noise and known parameters. Running the pipeline against it
    and recovering (r, mu) in analysis/ validates the whole loop end to end."""

    def __init__(self, r: float = 0.5, mu: float = 0.15, seed: int = 7):
        self.r, self.mu = r, mu
        self.rng = random.Random(seed)
        self.model = f"simulated-crra(r={r},mu={mu})"
        self.temperature = float("nan")

    def _u(self, x: float) -> float:
        if x <= 0:
            return 0.0
```

```
        return math.log(x) if abs(self.r - 1) < 1e-9 else x ** (1 - self.r) / (1 - self.r)

    def choose(self, trial: Trial) -> tuple[str, str]:
        eu_safe = self._u(trial.sure)
        eu_gamble = trial.p * self._u(trial.hi)
        # scale utilities so mu is comparable across stake levels
        scale = self._u(max(trial.hi, trial.sure))
        p_gamble = 1 / (1 + math.exp(-(eu_gamble - eu_safe) / (self.mu * scale)))
        chose_gamble = self.rng.random() < p_gamble
        letter = trial.gamble_letter if chose_gamble else ("A" if trial.gamble_letter == "B"
    ↪ else "B")
        return letter, f"simulated p_gamble={p_gamble:.3f}"
```

This class is a stand-in for the real model, and its answers are known by construction. It computes expected utility from a CRRA utility function with a fixed risk-aversion `r=0.5`, applies the same logit choice rule the estimator assumes, and flips a weighted coin to pick. Because I built it with $r = 0.5$ and $\mu = 0.15$, the estimator must recover those numbers if it is working. Notice that `choose` here has the exact same shape as `ClaudeAgent.choose`, it takes a trial and returns a (letter, raw) tuple, so the runner cannot tell the difference. That interchangeability is the whole trick: I can swap the fake decider in for the real one and exercise the entire pipeline without spending anything.

5.5 `run`: the loop that writes the CSV

```
def run(trials, agent, seed: int, out_path: Path, row_fn,
        max_retries: int = 3, sleep_s: float = 0.05) -> None:
    out_path.parent.mkdir(parents=True, exist_ok=True)
    new_file = not out_path.exists()
    n_done = n_discard = 0
    fieldnames = list(row_fn(trials[0]).keys()) + RUN_FIELDS

    with open(out_path, "a", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        if new_file:
            writer.writeheader()

        for i, t in enumerate(trials):
            letter, raw, latency = None, "", float("nan")
            for attempt in range(max_retries):
                t0 = time.time()
                try:
                    letter, raw = agent.choose(t)
                    latency = time.time() - t0
                    break
                except Exception as e:  # rate limits, transient API errors
                    raw = f"ERROR: {e}"
                    wait = 2 ** attempt
                    print(f"  trial {t.trial_id}: {e} (retry in {wait}s)", file=sys.stderr)
                    time.sleep(wait)

            discarded = letter not in ("A", "B")
            chose_gamble = "" if discarded else int(letter == t.gamble_letter)
            row = row_fn(t) | {
                "model": agent.model, "temperature": agent.temperature, "seed": seed,
```

```
                "letter": letter or "", "chose_gamble": chose_gamble,
                "raw": (raw or "")[:200], "discarded": int(discarded),
                "latency_s": round(latency, 3) if latency == latency else "",
                "ts_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
            }
            writer.writerow(row)
            f.flush()  # interrupted runs keep their data

            n_done += 1
            n_discard += int(discarded)
            if n_done % 25 == 0 or n_done == len(trials):
                print(f"{n_done}/{len(trials)} trials | discards: {n_discard}")
            time.sleep(sleep_s)
```

The `run` function loops over every trial and writes one CSV row per answer. Three robustness details are worth pointing out, because they are what let a long, paid run survive real-world hiccups.

Retry on error. The inner `for attempt in range(max_retries)` loop wraps the `choose` call in a `try/except`. If the API throws an error (a rate limit, a transient network failure), the code waits `2 ** attempt` seconds, one, then two, then four, and tries again. This is exponential backoff: a brief network stumble does not kill the run or lose a question.

Flush after every row. The line `f.flush()` forces the just-written row to disk immediately rather than sitting in a buffer. If the run is interrupted after 800 of 1,200 questions, all 800 answers are already safely on disk. The comment says it plainly: interrupted runs keep their data.

Discard tracking. A response is **discarded** if the letter is not a clean "A" or "B." The code records `discarded` as a flag and leaves `chose_gamble` blank for those rows, and it tallies the discard count so I can watch the rate. A high discard rate would signal that something is wrong with the elicitation, so it is surfaced immediately. This line,

```
chose_gamble = int(letter == t.gamble_letter)
```

is the moment a raw letter becomes the analysis variable: it is 1 exactly when the model's letter matches the gamble's letter for this trial.

> `run` walks the question list, calls the agent on each, and writes a fully-labeled CSV row per answer. It retries on API errors with growing waits, flushes each row to disk so an interrupted run keeps its data, and flags any answer that was not a clean A or B. The line that turns the model's letter into `chose_gamble` is where an answer becomes data.

6. Stage 3: Fitting the Math to the Answers (`estimate_crra.py`)

The CSV now holds a pile of 1s and 0s: for each question, did the model take the gamble? Stage 3 asks the reverse question. What risk attitude would make this exact pattern of 1s and 0s least surprising? That is the job of the estimator.

6.1 The utility function and the choice probability

```
def crra_u(x: np.ndarray, r: float) -> np.ndarray:
    x = np.maximum(x, 1e-12)
    if abs(r - 1.0) < 1e-9:
        return np.log(x)
    return x ** (1.0 - r) / (1.0 - r)


def choice_prob(df: pd.DataFrame, r: float, mu: float) -> np.ndarray:
    """Pr(choose gamble) for each row under (r, mu)."""
    eu_safe = crra_u(df["sure"].values.astype(float), r)
    eu_gamble = df["p"].values * crra_u(df["hi"].values.astype(float), r)
    scale = np.abs(crra_u(np.maximum(df["hi"].values, df["sure"].values).astype(float), r))
    z = (eu_gamble - eu_safe) / (mu * np.maximum(scale, 1e-12))
    return 1.0 / (1.0 + np.exp(-np.clip(z, -35, 35)))
```

These two functions are the theory from the teaching guide, written as code. `crra_u` is the CRRA utility function, exactly $u(x) = x^{1-r}/(1-r)$, with the standard special case that at $r = 1$ utility becomes the natural logarithm. The clamp `np.maximum(x, 1e-12)` just avoids taking a power of zero.

`choice_prob` is the logit choice rule. Line by line it mirrors the equation $P(\text{gamble}) = 1/(1 + e^{-(EU_g - EU_s)/(\mu \cdot \text{scale})})$. It computes the sure option's utility (`eu_safe`), the gamble's expected utility (`eu_gamble`, which is $p \times u(\text{hi})$), divides their difference by the noise μ times a scale factor, and passes that through the logistic curve `1/(1 + exp(-z))`. The output is one predicted probability of choosing the gamble for every row in the data. Feed it a candidate r and μ and it tells you how likely each observed choice was under those values. The `np.clip(z, -35, 35)` guards against overflow in the exponential; it does not change any realistic result.

6.2 Scoring a guess: negative log-likelihood

```
def neg_loglik(theta: np.ndarray, df: pd.DataFrame) -> float:
    r, log_mu = theta
    mu = np.exp(log_mu)  # mu > 0 via log transform
    pg = choice_prob(df, r, mu)
    y = df["chose_gamble"].values.astype(float)
    ll = y * np.log(np.clip(pg, 1e-12, 1)) + (1 - y) * np.log(np.clip(1 - pg, 1e-12, 1))
    return -ll.sum()
```

DEFINITION • Maximum likelihood

Maximum likelihood is a recipe for choosing parameter values. For any candidate values, the model assigns a probability to each choice the model actually made. Multiply those probabilities together (or, equivalently and more safely, add their logarithms) and you get one score for how unsurprising the observed choices are under those values. The maximum-likelihood estimate is the set of values that makes that score as high as possible: the values under which the real choices look as expected, as unsurprising, as possible.

`neg_loglik` computes that score, with two practical twists. First, it works with `log_mu` and immediately takes `mu = np.exp(log_mu)`; because the exponential of any number is positive, this guarantees the noise μ stays positive no matter what the search tries. Second, it returns the *negative* of the log-likelihood. The reason is purely mechanical: the optimizer in the next function is a minimizer, and minimizing the negative is the same as maximizing the original. The formula `ll = y * log(pg) + (1 - y) * log(1 - pg)` is the standard log-likelihood for a yes/no outcome: when the model chose the gamble (`y = 1`) it credits `log(pg)`, and when it chose the sure thing (`y = 0`) it credits `log(1 - pg)`. Summing across all rows and negating gives the number to minimize. In one sentence: `neg_loglik` measures how surprised a given (r, μ) is by the choices that were actually made.

6.3 Searching for the best fit

```
def estimate(df: pd.DataFrame) -> dict:
    best = None
    for r0 in (0.1, 0.5, 0.9, 1.5):  # multi-start; the likelihood can be flat in r
        res = optimize.minimize(neg_loglik, x0=np.array([r0, np.log(0.2)]),
                                args=(df,), method="Nelder-Mead",
                                options={"xatol": 1e-6, "fatol": 1e-8, "maxiter": 5000})
        if best is None or res.fun < best.fun:
            best = res

    r_hat, log_mu_hat = best.x
    mu_hat = float(np.exp(log_mu_hat))

    H = numerical_hessian(lambda th: neg_loglik(th, df), best.x)
    try:
        cov = np.linalg.inv(H)
        se_r = float(np.sqrt(max(cov[0, 0], 0)))
        # delta method for mu = exp(log_mu)
        se_mu = float(np.sqrt(max(cov[1, 1], 0)) * mu_hat)
    except np.linalg.LinAlgError:
        se_r = se_mu = float("nan")

    return {"r": float(r_hat), "se_r": se_r, "mu": mu_hat, "se_mu": se_mu,
            "loglik": -float(best.fun), "n": len(df), "converged": bool(best.success)}
```

DEFINITION • Optimizer

An **optimizer** is a search routine that hunts for the input that makes a function as small (or as large) as possible. Here `optimize.minimize` walks around the space of possible (r, μ) values, repeatedly calling `neg_loglik`, and homes in on the pair with the lowest surprise score. It is a systematic version of "try values, keep whichever fits best."

`estimate` runs the optimizer and packages the result. The `for r0 in (0.1, 0.5, 0.9, 1.5)` loop is a **multi-start**: it launches the search from four different starting points and keeps the best result. The reason is in the comment, the likelihood can be flat in r, meaning the surprise surface can have gentle regions where a search starting in the wrong place could stall. Starting from several places and keeping the best guards against getting stuck in one.

The last block turns the curvature of the surprise surface into a **standard error**.

DEFINITION • Standard error

A **standard error** is the give-or-take on an estimate: how much the estimated number would wobble if I reran the whole experiment. It comes from how sharply curved the surprise surface is at the best fit. A sharp, narrow minimum means the data pin the value down tightly (small standard error); a broad, flat minimum means many values fit almost as well (large standard error). The code measures that curvature with the **Hessian** (a matrix of second derivatives), inverts it to get the variance, and takes the square root.

The `numerical_hessian` call measures how fast the surprise score rises as I move away from the best fit in every direction. Inverting that matrix and taking square roots gives the standard errors `se_r` and `se_mu`. The comment `# delta method for mu = exp(log_mu)` flags a small correction: because the search worked in `log_mu` but I report `mu`, the standard error is scaled by `mu_hat` to translate the wobble back onto the natural scale.

`estimate` searches for the (r, μ) that best explain the choices, starting from several places so it does not get stuck, then reads the give-or-take on each number from how sharply the fit degrades nearby. It returns the risk-aversion estimate, the noise estimate, their standard errors, and whether the search converged.

6.4 The guardrails: sanity checks

Before trusting any estimate, the code runs a battery of checks. These are the referees that must pass before the numbers mean anything.

```
99  def sanity_checks(df: pd.DataFrame) -> None:
100     print("Sanity checks ")
101
102     # 1. Discard rate
103     if "discarded" in df.columns:
104         rate = df["discarded"].mean()
105         print(f"discard rate:        {rate:.1%}" + ("   FLAG (>2%)" if rate > 0.02 else "   ok
    ↪ "))
106
107     kept = df[df.get("discarded", 0) == 0].copy()
108     kept["chose_gamble"] = kept["chose_gamble"].astype(int)
109
110     # 2. Label/position balance: chose_gamble should not depend on where the safe
111     #    option sat. A significant gap = position bias masquerading as preference.
112     by_pos = kept.groupby("safe_first")["chose_gamble"].agg(["mean", "count"])
113     if len(by_pos) == 2:
114         a = kept[kept["safe_first"] == True]["chose_gamble"]
115         b = kept[kept["safe_first"] == False]["chose_gamble"]
116         _, pval = stats.ttest_ind(a, b)
117         gap = abs(a.mean() - b.mean())
118         print(f"position gap: ...")
119
```

```
    # 3. Monotonicity: P(gamble) should rise with the gamble's EV ratio.
    kept["ev_bin"] = pd.qcut(kept["ev_ratio"], 4, labels=False, duplicates="drop")
    means = kept.groupby("ev_bin")["chose_gamble"].mean()
    mono = means.is_monotonic_increasing

    # 5. Template spread (preview of the random-effects question)
    by_t = kept.groupby("template_id")["chose_gamble"].mean()
```

(The excerpt above trims the print formatting to keep the logic visible; the real file prints each result with an "ok" or "FLAG" label.) The checks, and why each one guards the result:

- **Discard rate.** If more than about 2 percent of answers were not a clean A or B, something is wrong with the elicitation and the code flags it. A clean run should discard almost nothing.
- **Position balance.** It compares gamble-taking when the safe option was listed first versus second. If those differ significantly, a position habit is leaking into the data and masquerading as a preference. The counterbalancing in Stage 1 is what makes this check possible.
- **Monotonicity.** It sorts gambles into four bins by how good a deal they are (`ev_ratio`) and checks that gamble-taking rises across the bins. If a decider takes better deals more often, this should hold; if it does not, the data are not behaving like choices at all.
- **Response-mode comparison.** It compares the forced-tool answers to the plain-text answers (the validation arm from Stage 1). If they agree, forcing the model to answer through a tool did not distort its choices.
- **Template spread.** It reports how much gamble-taking varies across the five wordings. A large spread would warn that the wording itself is doing a lot of work, which is exactly the kind of thing to know before interpreting the headline number.

`sanity_checks` is the pre-flight inspection. It confirms the model answered cleanly, that position did not sway it, that it took better deals more often, that the tool and text arms agree, and that no single wording dominates. Only if these pass do the fitted numbers deserve trust.

7. Stage 3, Continued: The Richer Model (`estimate_cpt.py`)

The plain CRRA model of Section 6 fit the Phase 1 data badly: the model chased win probability far more than plain risk aversion can explain. `estimate_cpt.py` adds the missing piece, probability weighting, and then formally compares models. I touch it lightly, because the machinery (negative log-likelihood, multi-start optimizer, Hessian standard errors) is the same as before. The new ingredient is one function.

7.1 The Prelec probability-weighting function

```
def prelec_w(p: np.ndarray, gamma: float) -> np.ndarray:
    p = np.clip(p, 1e-9, 1 - 1e-9)
    return np.exp(-((-np.log(p)) ** gamma))
```

This is the Prelec weighting function $w(p) = \exp(-(-\ln p)^\gamma)$, one line of code. It converts a real, stated probability `p` into a *felt* probability that the decider acts on. When $\gamma = 1$ it does nothing (felt equals real). When $\gamma > 1$ it bends into an S-shape that crushes small probabilities toward zero, so long shots feel even more hopeless than they are, which is exactly the Phase 1 finding. The `np.clip` just keeps the logarithm away from the forbidden values 0 and 1.

7.2 Three models, one contest

The richer model uses `prelec_w` in place of the raw probability. It is compared against two others:

```
78 MODELS = {
79     "M1 CRRA":            (prob_m1, ["r", "log_mu"],                      [0.3, np.log(0.2)]),
80     "M2 CRRA+Prelec+pos": (prob_m2, ["r", "log_gamma", "log_mu", "delta"],[0.3, 0.3, np.log
       ↪ (0.2), 0.5]),
81     "M3 p-only":          (prob_m3, ["a", "b"],                           [-4.0, 8.0]),
82 }
```

The three contestants are: **M1**, the plain CRRA plus logit from Section 6; **M2**, the same plus Prelec probability weighting and a position term (the improved model); and **M3**, a stripped "probability heuristic" that ignores the payoffs entirely and predicts choice from the win probability alone. The script fits all three, prints a comparison table with each model's fit and its AIC and BIC (standard penalties for complexity), and runs a likelihood-ratio test of M1 against M2.

```
161     m1, m2 = results["M1 CRRA"], results["M2 CRRA+Prelec+pos"]
162     lr = 2 * (m2["ll"] - m1["ll"])
163     p_lr = stats.chi2.sf(lr, df=m2["k"] - m1["k"])
```

The likelihood-ratio statistic `lr = 2 * (m2["ll"] - m1["ll"])` measures how much better the richer model M2 explains the data than the plain M1. In the Phase 1 data the improvement was overwhelming: adding probability weighting wins massively, and the "p-only" M3 nearly ties the full structural model, both signs that the model is pricing probability rather than payoffs.

`estimate_cpt.py` adds one function, `prelec_w`, that bends a real probability into a felt one, and stages a three-way contest: plain risk aversion, risk aversion plus probability weighting, and a probability-only heuristic. The probability-weighting model wins by a wide margin, which is the quantitative core of the Phase 1 headline.

8. Follow One Choice All the Way Through

To tie the three stages into a single story, here is the life of one choice, from a row in a list to a term in a likelihood.

1. **Design.** `build_trials` creates a `Trial`, say gamble 12, template 0, safe listed first, repetition

1. At this point it is just a labeled row of numbers in a Python list.

2. **Wording.** The harness calls `trial.prompt()`. `Trial.prompt` builds "receive $58 with certainty" and "receive $116 with 66% probability, and $0 with 34% probability," puts the safe one first (because `safe_first` is true), and slots them into template 0. Out comes the exact question text.

3. **Ask.** `ClaudeAgent.choose` sends that text to the model as a fresh, one-message conversation with the choice tool attached. The model answers "B."

4. **Record.** Back in `run`, the code compares "B" to `trial.gamble_letter`. Because the safe option was first, the gamble is "B," so the letters match and `chose_gamble = 1`. That 1, along with `sure`, `hi`, `p`, and every design label, is written as one CSV row and flushed to disk.

5. **Fit.** Much later, `neg_loglik` reads that row. For a candidate (r, μ), `choice_prob` predicts some probability `pg` that the gamble would be chosen here. Because this row has `chose_gamble = 1`, the row contributes `log(pg)` to the score. The optimizer nudges r and μ until the total surprise across all rows, this one included, is as small as possible.

One choice, five steps, three files. That is the entire experiment in miniature.

9. Try It Yourself

Everything in Sections 3 through 7 can be run without an API key, because of the dry-run path. Here is the minimum that a reader can execute right now.

```
# 1. Ask a known fake decider 1,200 questions and save the answers.
python src/harness.py --phase 1 --dry-run --reps 3

# 2. Fit the model to those answers and read the estimates back.
python analysis/estimate_crra.py data/phase1_dryrun.csv
```

The first command builds the full question list, runs it against `SimulatedCRRAAgent` (the fake decider built with $r = 0.5$ and $\mu = 0.15$), and writes `data/phase1_dryrun.csv`. The second command reads that CSV, runs the sanity checks, and fits the CRRA model. What you should see is the estimator recovering the fake decider's known settings: an r near 0.5 and a μ near 0.15, with the sanity checks passing. That recovery is the proof that the pipeline is wired correctly, and it is exactly the rehearsal I run before ever spending a cent on the live model.

If you want to see the Phase 2 wording with your own eyes, run `python src/design_phase2.py` and read the four printed prompts. The mixed frame is the one that says "loss."

Every code block in this walkthrough is copied verbatim from the repository as of July 2026: `src/design.py`, `src/design_phase2.py`, `src/harness.py`, `analysis/estimate_crra.py`, and `analysis/estimate_cpt.py`. Where a block was trimmed for readability (only in the sanity-check listing), the omission is noted in the surrounding text and no logic was altered.